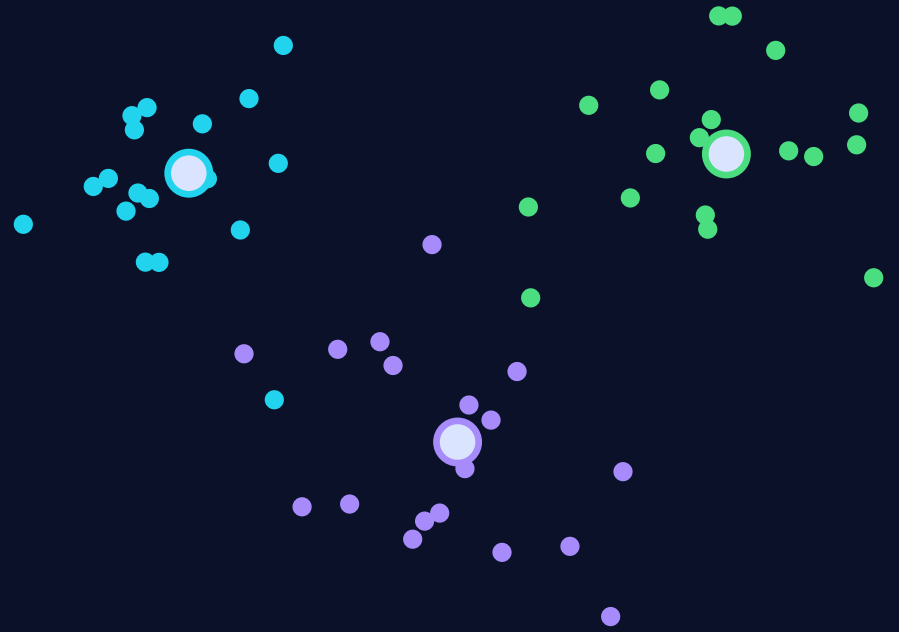
Guía de estudio · Módulo 27

El algoritmo K-Means

Del notebook a la explicación: qué hace cada paso, cómo leer cada gráfica y qué errores evitar.



150 flores

4 variables

3 clústers

95.8 % varianza en 2D

Mapa del notebook

Teoría en las diapositivas 3 y 4; después, un paso del notebook por diapositiva.

1

Datos

diap. 5

2

K-Means por dentro

diap. 6

3

Elegir k

diap. 7

4

Validar k = 3

diap. 8

5

Escalar y PCA

diap. 9

6

Agrupar en PCA

diap. 10

7

Comparar

diap. 11

8

Pro, glosario, repaso

diap. 12 a 14

Algoritmo de Lloyd e inercia

algoritmo_lloyd.py

inicializar # k-means++

Se eligen k centroides; k-means++ los reparte separados. La versión clásica los toma al azar.

asignar # paso 2

Cada punto va al centroide más cercano (distancia euclidiana).

actualizar # paso 3

Cada centroide pasa a ser el promedio de sus puntos. Se repite hasta que nada cambia.

inercia_y_optimo_local.md

inercia suma de distancias al cuadrado de cada punto a su centroide; es lo que K-Means minimiza

óptimo local cada iteración mejora la inercia, pero puede quedar en una solución peor que la mejor

n_init ejecuta varias inicializaciones y conserva la de menor inercia

Elegir k y conocer los límites

elegir_k.py

codo # inercia vs k

La inercia siempre baja al subir k; se busca dónde la mejora se modera. Lectura visual, a veces ambigua.

silueta # s = (b - a) / max(a, b)

De −1 a 1: cohesión del punto frente a la separación del clúster vecino. Favorece grupos compactos.

decisión # no hay k óptimo

Combina codo, silueta y criterio de dominio, y explica la elección.

supuestos.md

forma clústers aproximadamente esféricos y de tamaño parecido

escala sensible a la escala de las variables y a los atípicos

k fijo se elige de antemano y el resultado depende de la inicialización

Paso 1 · datos

Cargar y auditar Iris

celdas_1_a_3.py

```
df = pd.read_csv("iris.csv")   # o la URL de Seaborn si no existe
df.columns = [c.replace(".", "_").lower() for c in df.columns]
X_raw = df.drop(columns="species")   # y_real solo para evaluar
```

150 x 4

flores x medidas (cm)

1

fila duplicada (normal en Iris)

Cómo leerlo

Las 4 variables están en cm y en rangos comparables (0.1 a 7.9). La especie real no se usa para agrupar.

Error común

Agrupar usando la columna species: sería aprendizaje supervisado.

Paso 2 · por dentro

K-Means desde cero

kmeans_desde_cero.py

```
centroides = X[rng.choice(len(X), size=k, replace=False)]   # inicializar
etiquetas = dist.argmin(axis=1)   # asignar al centroide más cercano
nuevos = [X[etiquetas == j].mean(axis=0) for j in range(k)]   # actualizar
```

78.85 — mejor inercia (n_init = 10)

145.45 — peor con init aleatoria y n_init = 1

1 solución — con k-means++ en 30 semillas

Cómo leerlo

Con inicialización aleatoria, algunas semillas terminan en inercia 142 a 145 en vez de 78.85. Con k-means++ las 30 semillas llegan a ≈ 78.85.

Error común

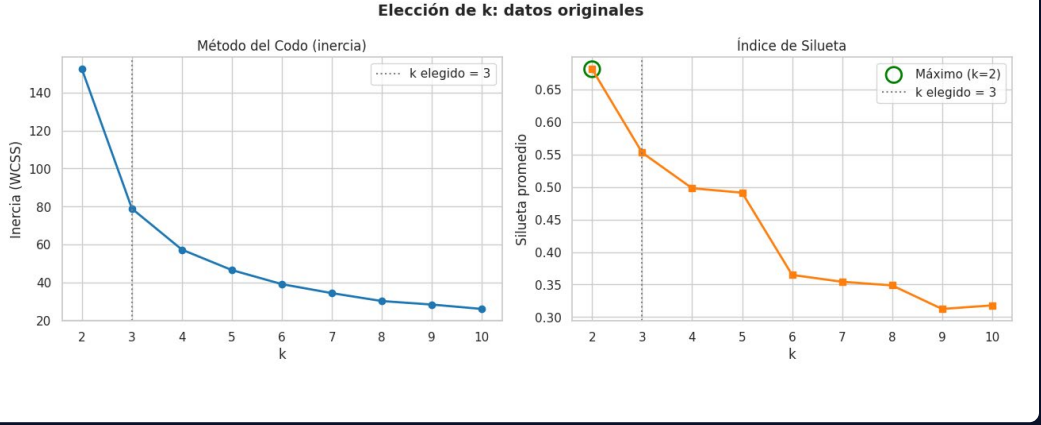
Reportar una sola ejecución aleatoria sin n_init.

Paso 3 · elegir k

Codo y silueta (datos originales)

celdas_6_a_8.py

```
res = evaluar_k(X_raw, ks=range(2, 11))   # inercia y silueta por k
silhouette_score(X, KMeans(k, n_init=10, random_state=42).fit_predict(X))
```



Cómo leerlo

Codo: inflexión en k = 3. Silueta: máximo en k = 2 (0.68); en k = 3 baja a 0.55. Con k = 2 se separa Setosa del resto.

Error común

Presentar k = 3 como "el óptimo": se eligió con codo, silueta y contexto (3 especies).

Paso 4 · validar

K-Means con k = 3 y validación externa

celdas_9_y_10.py

```
etq_raw = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(X_raw)
pd.crosstab(y_real, etq_raw)   # validación externa
adjusted_rand_score(y_real, etq_raw)   # 0.73
```

ARI 0.73 — vs. especie real

16 — puntos fuera de su especie dominante

0.55 vs 0.50 — silueta: K-Means vs. especies

Cómo leerlo

Setosa queda pura; el error está entre Versicolor y Virginica. La silueta de K-Means (0.55) supera a la de las especies reales (0.50).

Error común

Confundir silueta alta con coincidir con las clases, o interpretar el número de clúster (es arbitrario).

Dos transformaciones distintas

celdas_11_y_12.py

```
X_scaled = StandardScaler().fit_transform(X_raw)   # 1. cambia el peso
pca = PCA(n_components=2)   # 2. rota y descarta
X_pca = pca.fit_transform(X_scaled)
```

95.8 % varianza en 2 componentes

73.0 % PC1

22.9 % PC2

Cómo leerlo

PC1 y PC2 conservan 95.8 %: el 2D es fiel. PC1 ≈ tamaño (largo de sépalo y pétalos); PC2 ≈ ancho del sépalo.

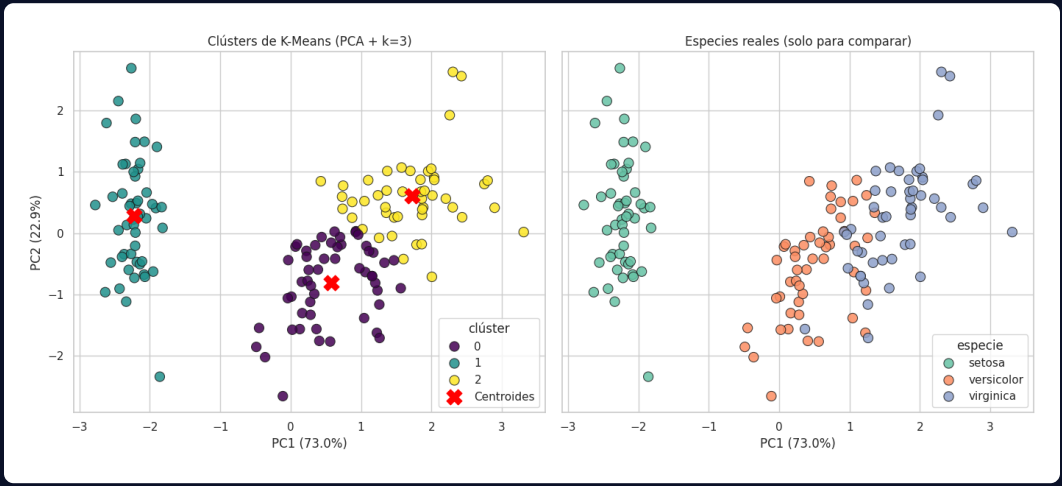
Error común

Aplicar escalado y PCA juntos y atribuir todo el cambio al PCA.

Repetir k y agrupar en 2D

celdas_13_y_14.py

```
res_pca = evaluar_k(X_pca)   # silueta máx. en k = 2 (0.62); k = 3: 0.51
kmeans_pca = KMeans(n_clusters=3, n_init=10, random_state=42)
etq_pca = kmeans_pca.fit_predict(X_pca)
```



Cómo leerlo

Mismo patrón que con los datos originales. El gráfico compara los clústers (izquierda) con las especies reales (derecha).

Error común

Fijar k = 3 sin repetir codo y silueta en el espacio PCA.

Comparación cuantitativa

celdas_15_y_16.py

```
etq_esc = KMeans(3, n_init=10, random_state=42).fit_predict(X_scaled)   # sin PCA
adjusted_rand_score(etq_esc, etq_pca)   # 1.0
```

0.73 → 0.62
ARI vs. especie: originales → escalados o PCA

1.0
ARI entre escalados (4D) y PCA (2D)

16 → 25
puntos fuera de su especie dominante

Cómo leerlo

El PCA de 2 componentes no cambió nada frente a solo estandarizar. La diferencia con la versión original viene de escalar.

Error común

Afirmar que el PCA limpia el ruido: aquí coincide menos con las especies (0.62 vs 0.73).

Nivel Pro

Pipeline con fit, predict y n_init

clase_pro.py

```
modelo = KMeansRecommenderPro(n_clusters=3, use_pca=True).fit(X_raw)
modelo.predict(flor_nueva)   # asigna clúster a datos nuevos
KMeans(n_clusters=3, n_init=10)   # no n_init="auto" (1 ejecución)
```

115.02 — inercia con n_init = 10

166.04 — inercia con n_init = "auto"

ARI 0.43 — asignaciones vs. análisis manual

Cómo leerlo

El pipeline reproduce el análisis manual (ARI 1.0) y puede asignar flores nuevas.

Error común

n_init="auto" hace una sola ejecución: inercia 166.04 en vez de 115.02 y ARI 0.43 contra el análisis manual.

Glosario

Centroide

promedio de los puntos de un clúster

Inercia (WCSS)

suma de distancias al cuadrado a los centroides; lo que K-Means minimiza

k-means++

inicialización que reparte los centroides iniciales de forma separada

n_init

número de inicializaciones; se conserva la de menor inercia

Óptimo local

solución que no mejora con cambios pequeños, pero no es la mejor posible

Codo

gráfico de inercia vs. k; se busca donde la mejora se modera

Silueta

de −1 a 1: qué tan bien encaja un punto en su clúster

ARI

1 = agrupaciones idénticas; 0 = azar

Estandarización

media 0 y desviación 1 por variable

Pipeline

secuencia de transformaciones y modelo aplicada igual a datos nuevos

Repaso y ejercicios

repaso.py

?> ¿Qué minimiza K-Means? La inercia: suma de distancias al cuadrado de cada punto a su centroide.

?> ¿Por qué usar n_init mayor que 1? Cada ejecución puede quedar en un óptimo local; se conserva la de menor inercia.

?> ¿Por qué la silueta prefiere k = 2 en Iris? Versicolor y Virginica se traslapan: hay dos bloques bien separados.

?> ¿El PCA mejoró la agrupación? No: con 2 componentes da lo mismo que solo estandarizar (ARI 1.0).

?> ¿Qué pasa con n_init="auto"? Hace una sola ejecución: puede dar una solución peor y distinta a la de n_init = 10.

ejercicios.md

1. Repite con k = 2 y k = 4: ¿qué clúster se divide?
2. Corre kmeans_desde_cero con 100 semillas: ¿qué porcentaje llega a 78.85?
3. Agrupa solo con petal_length y petal_width y compara el ARI.